

DevOps Capstone Project:

Deployment of Online Food Ordering and Delivery Application

Submitted to - Vikul Sir

Date of Submission – 27/MAY/2026

Submitted by – Rahul Nalathathi

Project Overview:

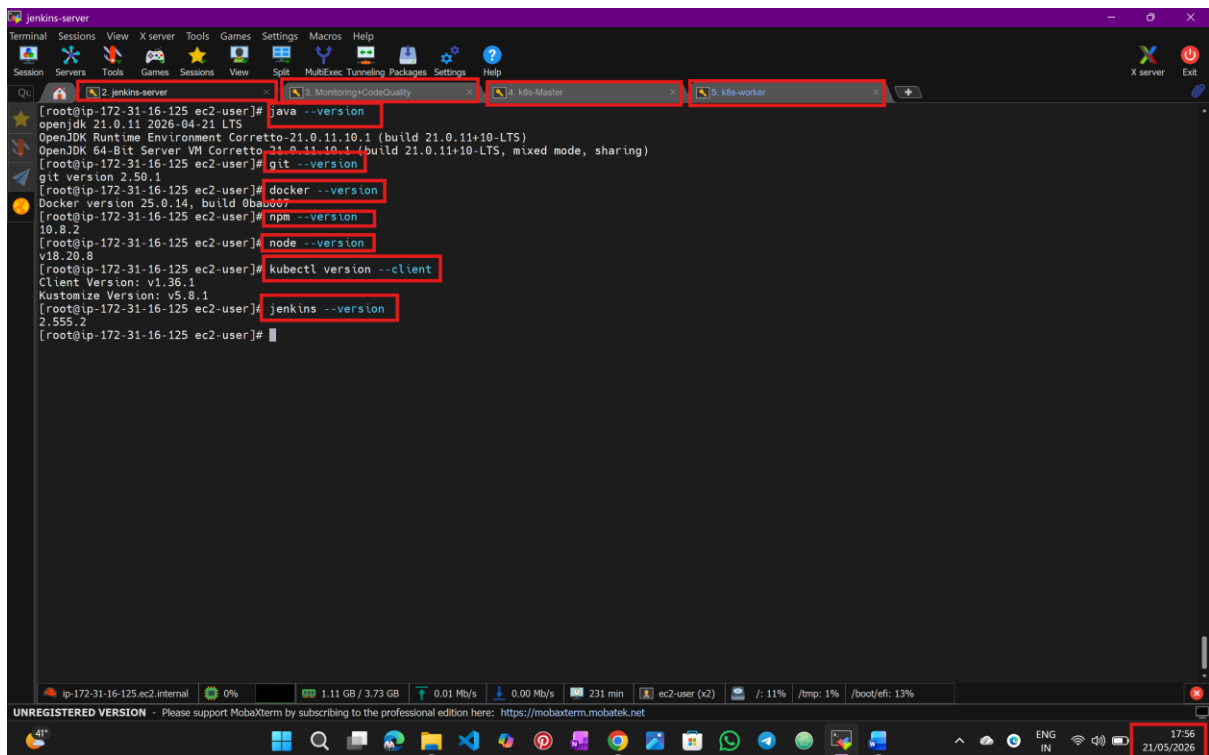
Zomato is a leading online food delivery and restaurant discovery platform. To handle increasing user demand and improve deployment efficiency, the company is transitioning from a monolithic architecture to a microservices-based architecture using DevOps practices. The goal is to implement a CI/CD pipeline, containerized deployments, and real-time monitoring to ensure high availability, faster releases, and automated scalability. AWS will be used as the primary cloud provider for hosting and deployment.

Step:1 Login into AWS console and Create 4 ec2 instances

The screenshot displays the AWS Management Console for the 'us-east-1' region. The 'Instances' page shows a list of 4 EC2 instances. A red box highlights the instance list table. Below the table, the 'Monitoring' section shows four graphs: CPU utilization (%), Network in (bytes), Network out (bytes), and Network packets in (count). The bottom right corner of the screenshot shows the system clock as 17:55 on 21/05/2026.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
jenkins-server	i-085fdf22778af8149	Running	c7i-flex.large	Initializing	us-east-1c	ec2-54-221-30-72.com...	54.221.30.72
Monitoring+CodeQuality	i-0ceb7ab9bb0f581f	Running	c7i-flex.large	Initializing	us-east-1c	ec2-54-166-161-192.co...	54.166.161.192
k8s-master	i-060e0b8799a213ccd	Running	c7i-flex.large	Initializing	us-east-1c	ec2-54-207-60-43.com...	54.207.60.43
k8s-worker	i-00e921ff92af39508	Running	c7i-flex.large	Initializing	us-east-1c	ec2-98-93-246-155.co...	98.93.246.155

Step:2 connect all with SSH via MobaXterm and install prerequisite in all of them.



The screenshot shows a MobaXterm terminal window with multiple tabs. The active tab is 'jenkins-server'. The terminal output shows the following commands and their results:

```
[root@ip-172-31-16-125 ec2-user]# java --version
openjdk 21.0.11 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-21.0.11.10-LTS (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.11.10-LTS (build 21.0.11+10-LTS, mixed mode, sharing)

[root@ip-172-31-16-125 ec2-user]# git --version
git version 2.50.1

[root@ip-172-31-16-125 ec2-user]# docker --version
Docker version 25.0.14, build 0ba0007

[root@ip-172-31-16-125 ec2-user]# npm --version
10.8.2

[root@ip-172-31-16-125 ec2-user]# node --version
v18.20.8

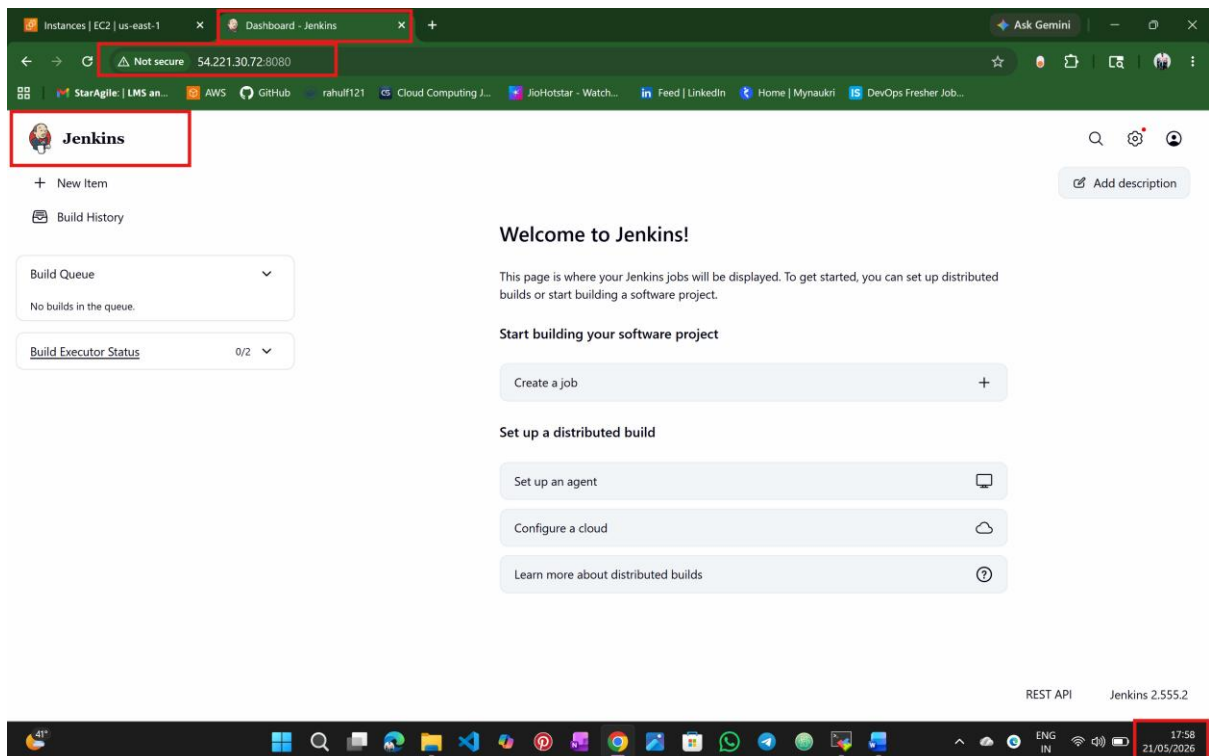
[root@ip-172-31-16-125 ec2-user]# kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1

[root@ip-172-31-16-125 ec2-user]# jenkins --version
2.555.2

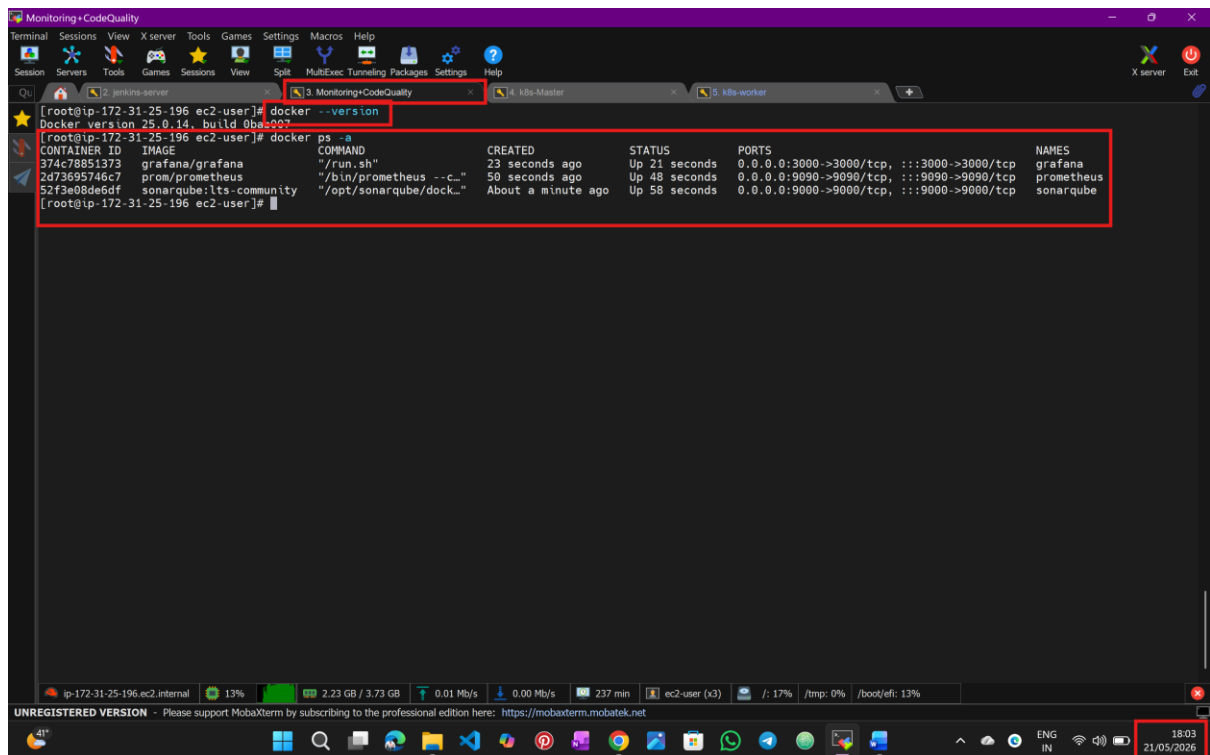
[root@ip-172-31-16-125 ec2-user]#
```

The terminal window also shows the status bar at the bottom with the text 'UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net' and the system tray showing the time as 17:56 on 21/05/2026.

Step:3 Access the Jenkins website



Step:4 install SonarQube, Prometheus and Grafana via Docker run



The screenshot shows a MobaXterm terminal window with the following content:

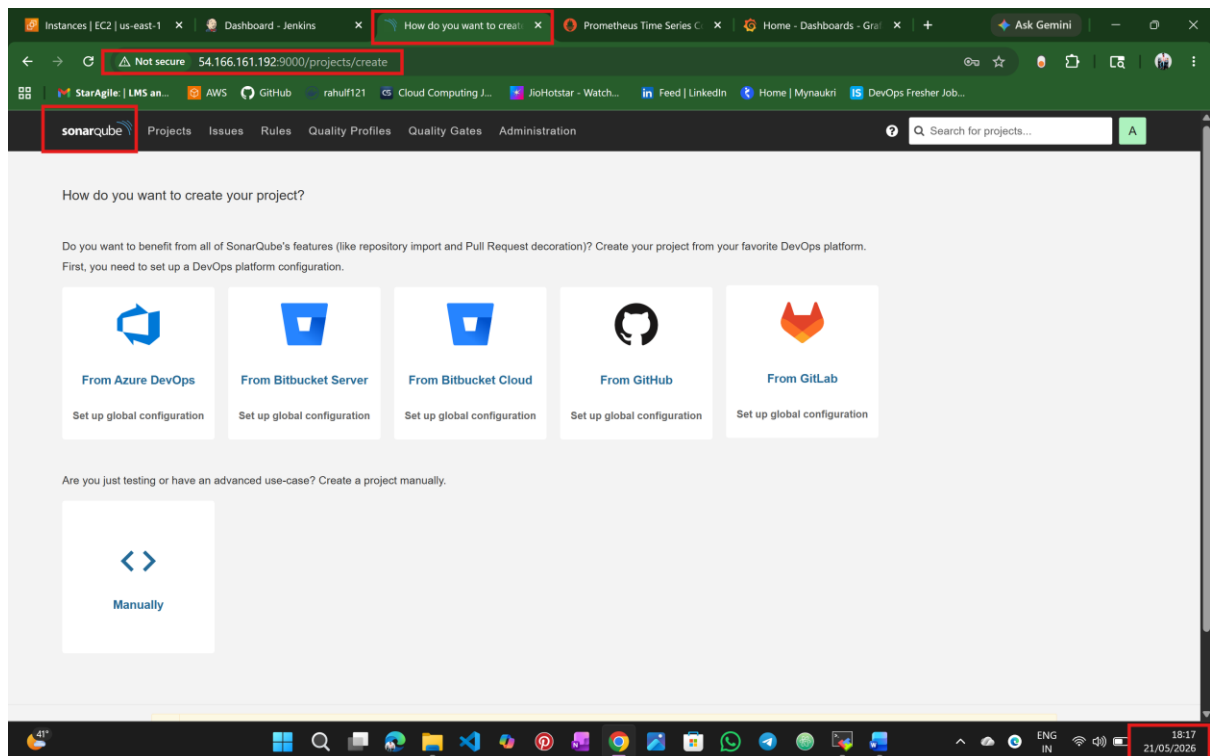
```
[root@ip-172-31-25-196 ec2-user]# docker --version
Docker version 25.0.14, build 0bae9007

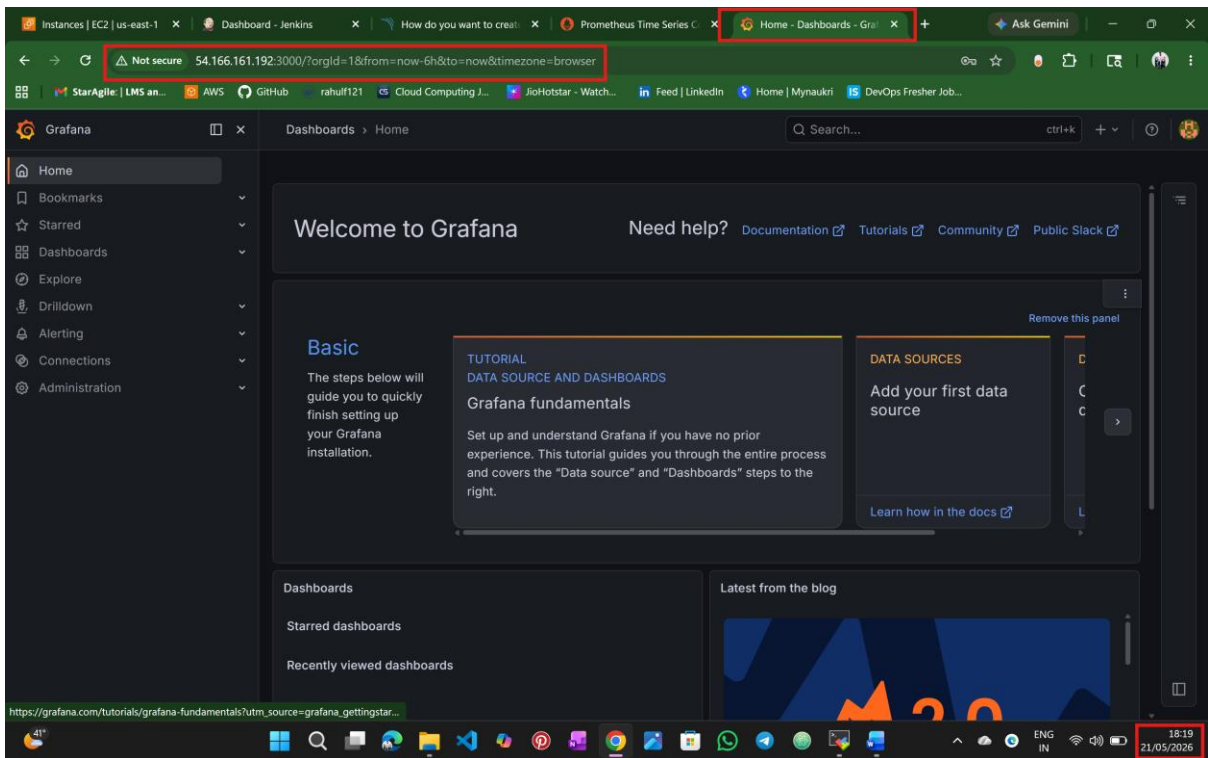
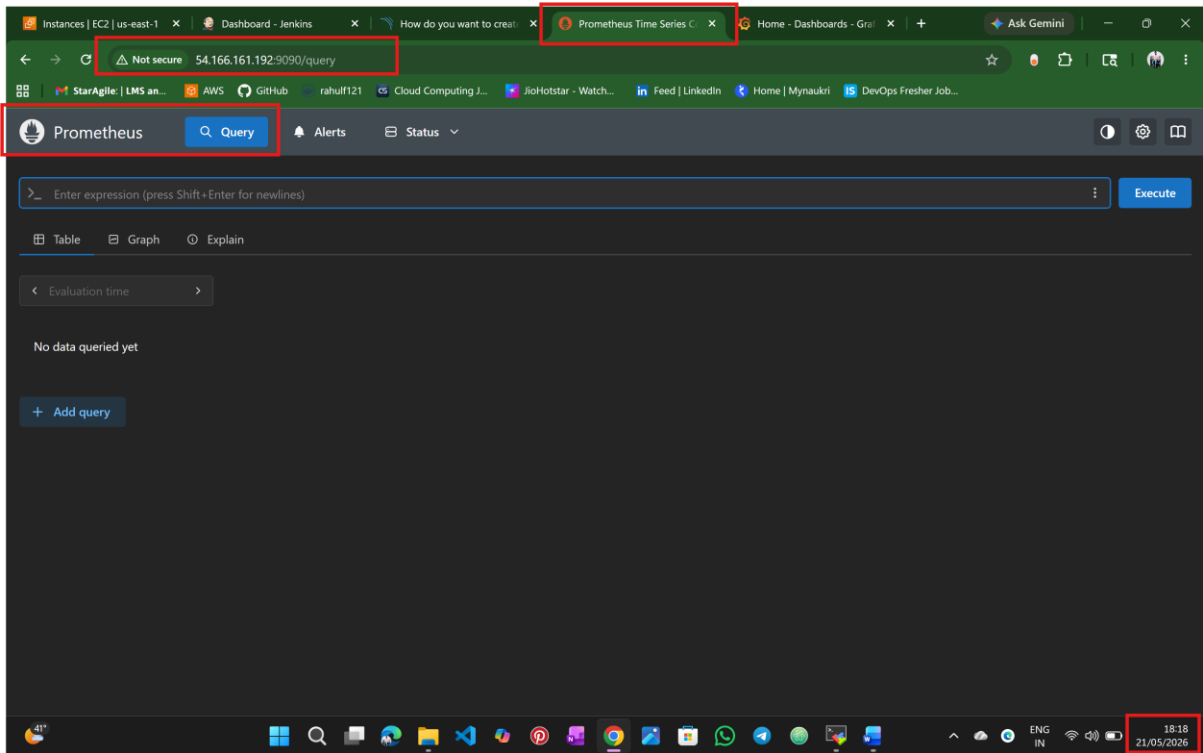
[root@ip-172-31-25-196 ec2-user]# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
374c78851373	grafana/grafana	"/run.sh"	23 seconds ago	Up 21 seconds	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp	grafana
2d73695746c7	prom/prometheus	"/bin/prometheus --c..."	58 seconds ago	Up 48 seconds	0.0.0.0:9090->9090/tcp, :::9090->9090/tcp	prometheus
52f3e09de5df	sonarqube/lts-community	"/opt/sonarqube/dock..."	About a minute ago	Up 58 seconds	0.0.0.0:9000->9000/tcp, :::9000->9000/tcp	sonarqube

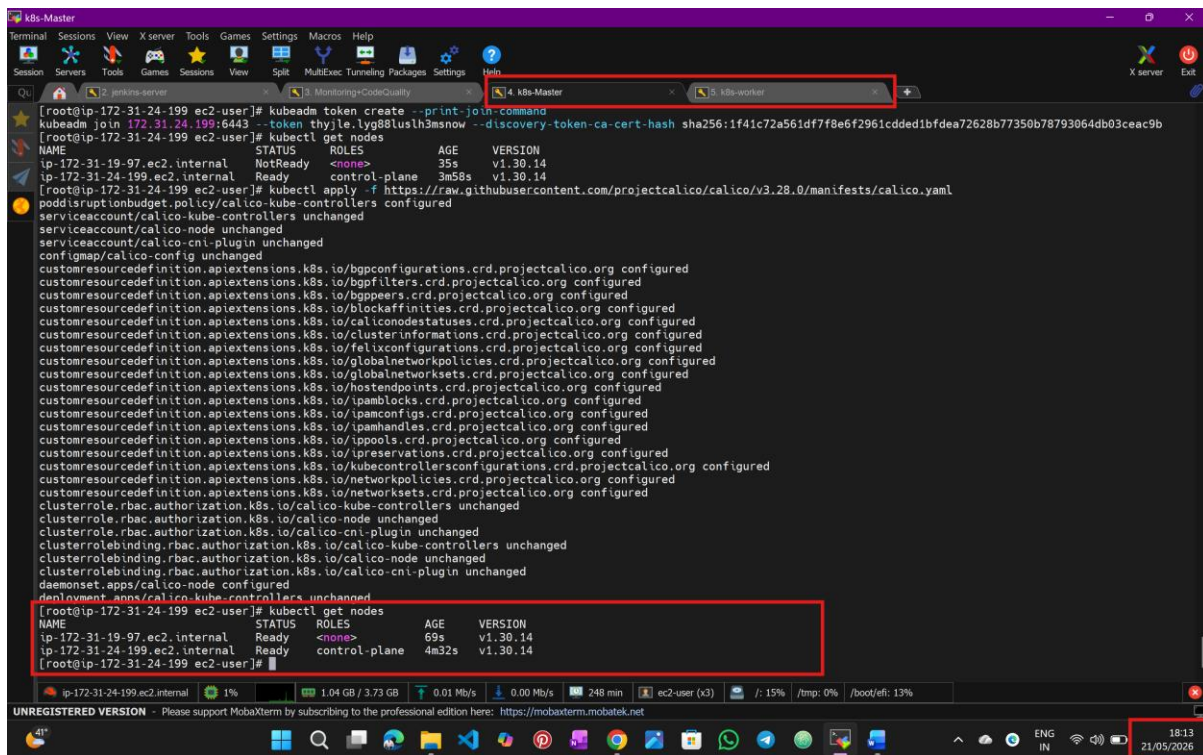
The terminal window also shows system information at the bottom: ip-172-31-25-196.ec2.internal, 13% CPU, 2.23 GB / 3.73 GB memory, 0.01 MB/s network, 0.00 MB/s disk, 237 min uptime, and 18:03 21/05/2026.

Step: 5 Access the SonarQube website, Prometheus and Grafana



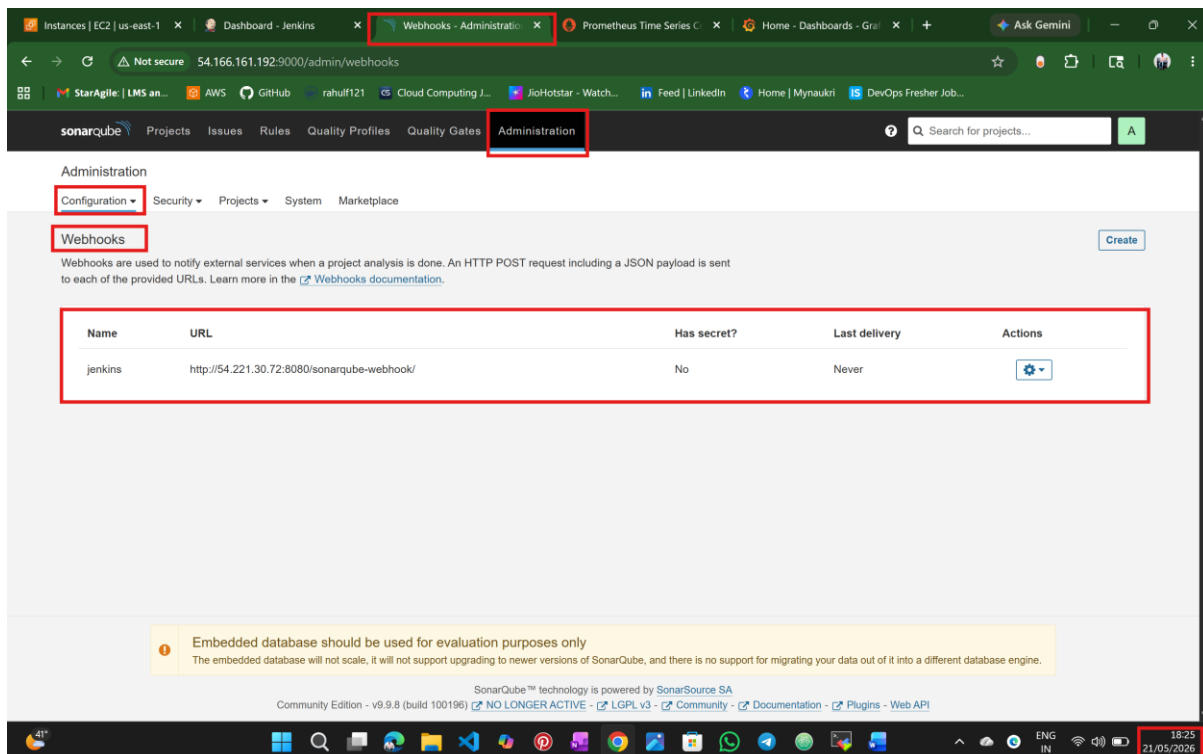


Step:6 connect K8s master and k8s worker nodes



```
[root@ip-172-31-24-199 ec2-user]# kubeadm token create --print-join-command
kubeadm join 172.31.24.199:6443 --token thylt6.lyg88lsh3msnow --discovery-token-ca-cert-hash sha256:1f41c72a561df7f8e6f2961cdded1bfdea72628b77350b78793064db03ceac9b
[root@ip-172-31-24-199 ec2-user]# kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-172-31-19-97.ec2.internal        NotReady <none>    35s     v1.30.14
ip-172-31-24-199.ec2.internal        Ready     control-plane 3m58s    v1.30.14
[root@ip-172-31-24-199 ec2-user]# kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers configured
serviceaccount/calico-kube-controllers unchanged
serviceaccount/calico-node unchanged
configmap/calico-cni-plugin unchanged
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/bgpfilters.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/l3preservations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org configured
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers unchanged
clusterrole.rbac.authorization.k8s.io/calico-node unchanged
clusterrolebinding.rbac.authorization.k8s.io/calico-cni-plugin unchanged
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers unchanged
clusterrolebinding.rbac.authorization.k8s.io/calico-node unchanged
daemonset.apps/calico-node configured
deployment.apps/calico-kube-controllers unchanged
[root@ip-172-31-24-199 ec2-user]# kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-172-31-19-97.ec2.internal        Ready     <none>    69s     v1.30.14
ip-172-31-24-199.ec2.internal        Ready     control-plane 4m32s    v1.30.14
[root@ip-172-31-24-199 ec2-user]#
```

Step:7 connect SonarQube and Jenkins using webhook



Administration

Configuration Security Projects System Marketplace

Webhooks

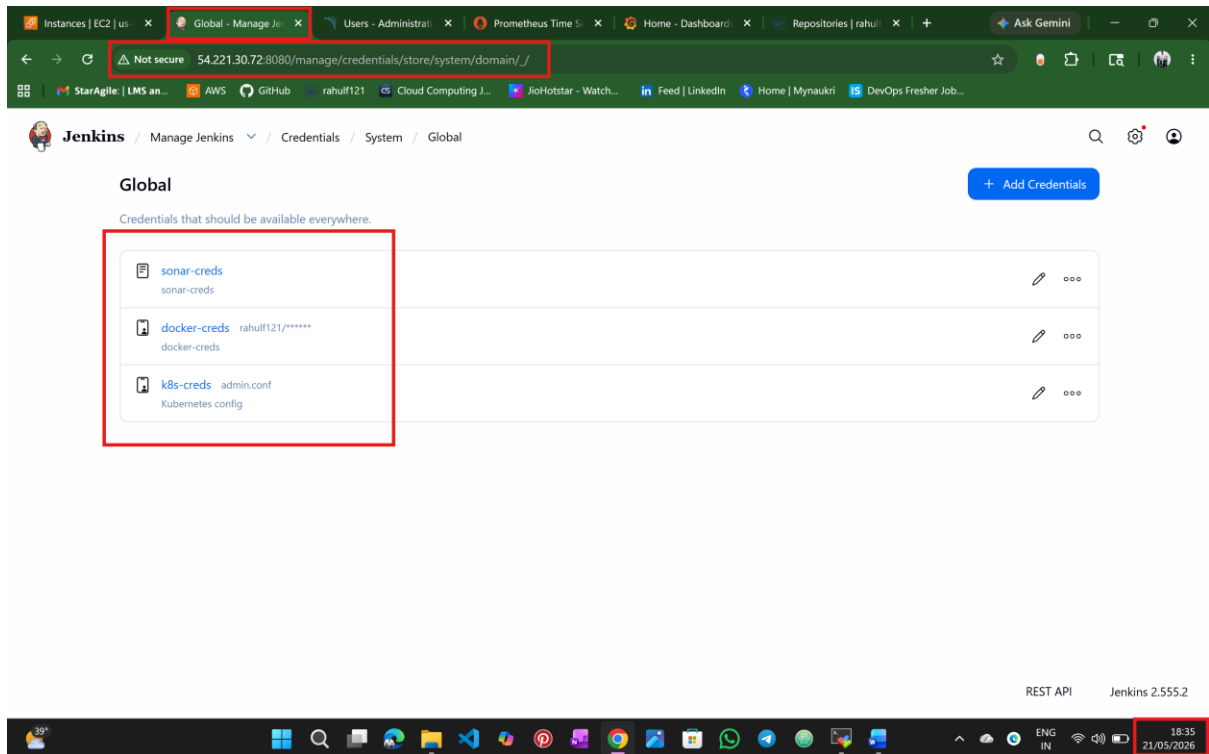
Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the [Webhooks documentation](#).

Name	URL	Has secret?	Last delivery	Actions
jenkins	http://54.221.30.72:8080/sonarqube-webhook/	No	Never	Settings

Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) [NO LONGER ACTIVE](#) [LGPL v3](#) [Community](#) [Documentation](#) [Plugins](#) [Web API](#)

Step:8 store the credentials in jenkins

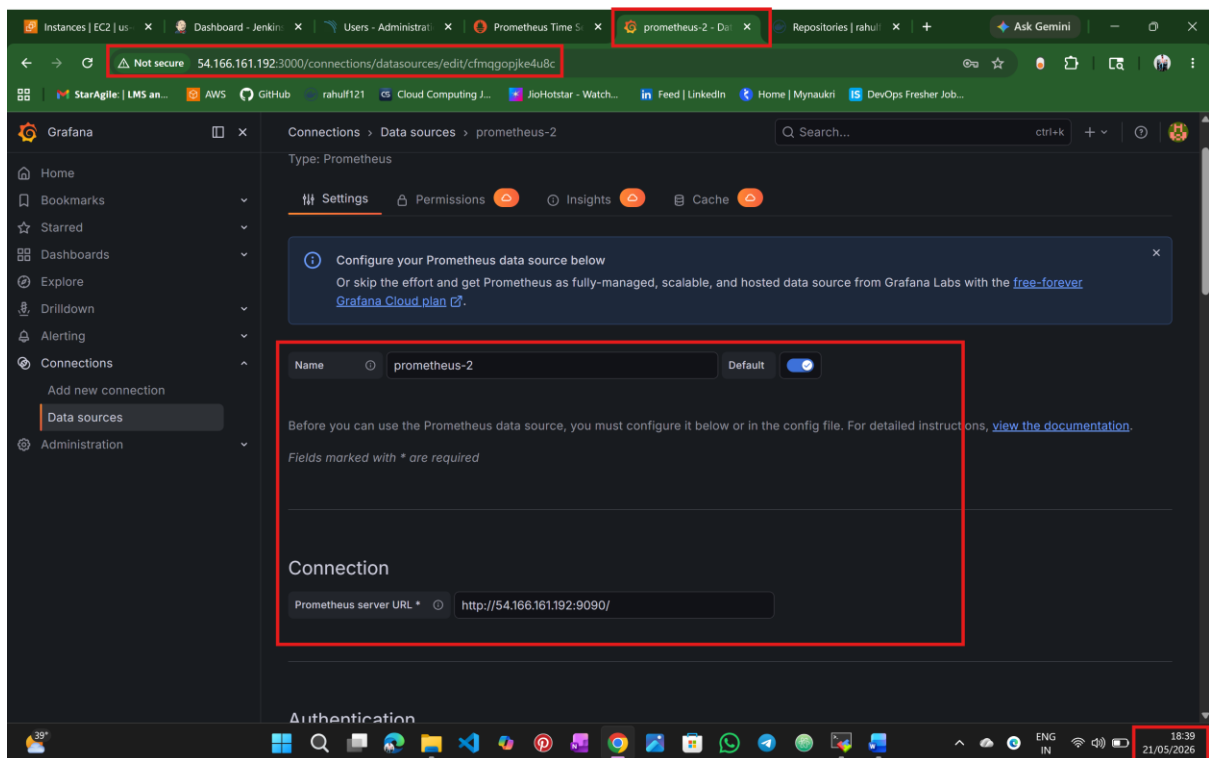


The screenshot shows the Jenkins web interface. The browser's address bar is highlighted with a red box, showing the URL: `54.221.30.72:8080/manage/credentials/store/system/domain/`. The page title is "Global". Below the title, there is a list of credentials. A red box highlights the following credentials:

Name	Username	Kind
sonar-creds	sonar-creds	sonar-creds
docker-creds	rahul121/*****	docker-creds
k8s-creds	admin.conf	Kubernetes config

The bottom of the screenshot shows a Windows taskbar with the system clock displaying 18:35 on 21/05/2026.

Step:9 connect Prometheus and Grafana



The screenshot shows the Grafana web interface. The browser's address bar is highlighted with a red box, showing the URL: `54.166.161.192:3000/connections/datasources/edit/cfmqgopjke4u8c`. The page title is "prometheus-2". The "Settings" tab is selected. A red box highlights the "Connection" section, which contains the following information:

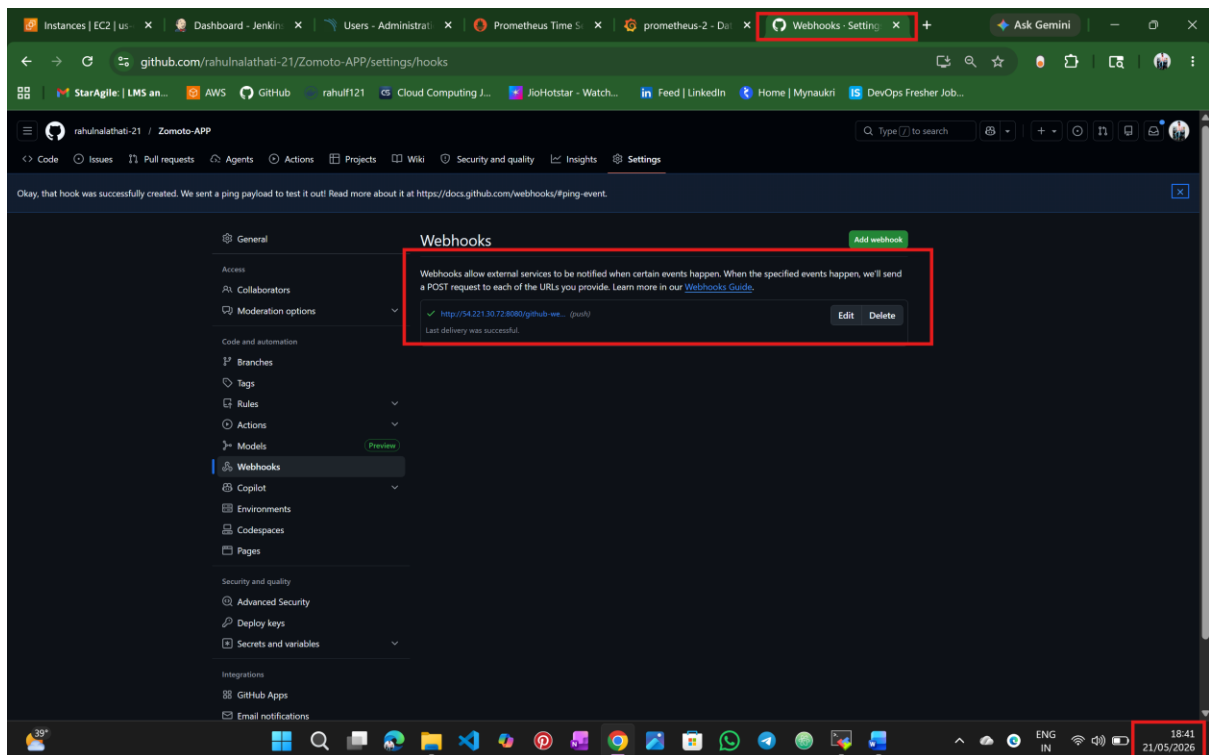
Name: prometheus-2

Default: ☒

Prometheus server URL: `http://54.166.161.192:9090/`

The bottom of the screenshot shows a Windows taskbar with the system clock displaying 18:39 on 21/05/2026.

Step:10 connect git hub and Jenkins using webhook



Step:11 build the pipeline

The screenshot shows the Jenkins pipeline for 'zomato-APP'. The pipeline is currently in a failed state. The build stages and their durations are as follows:

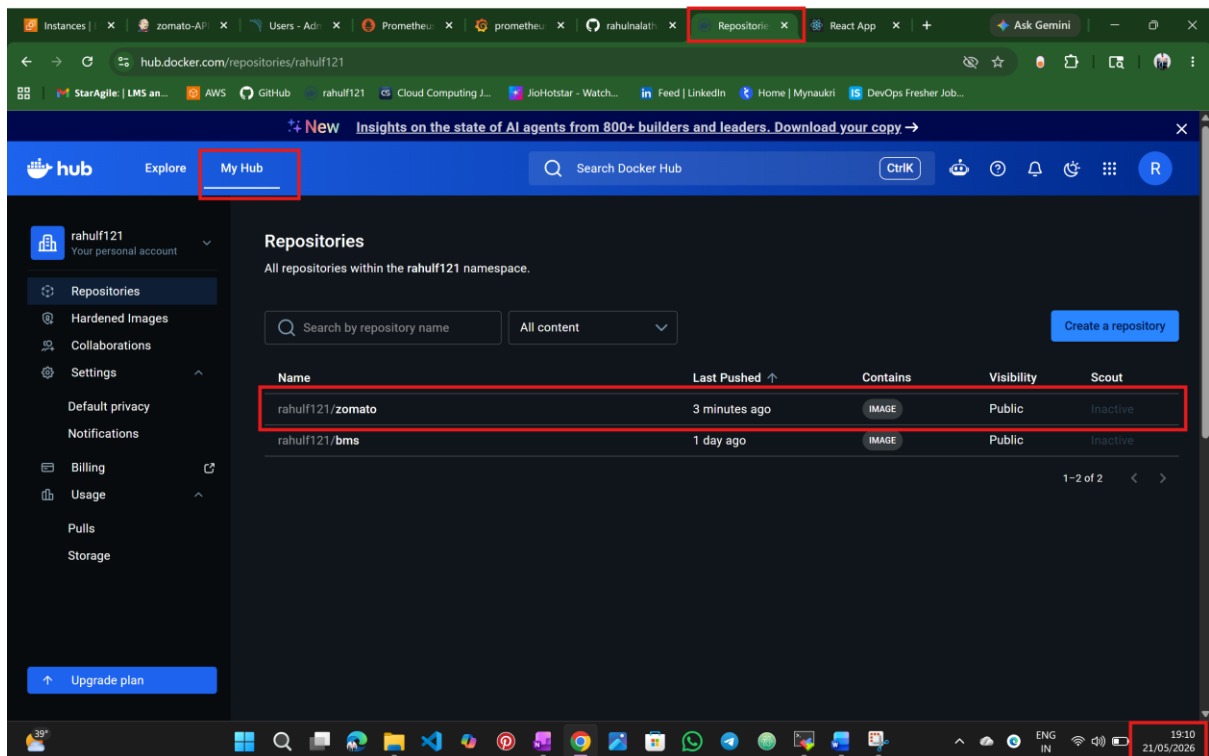
Clone Repository	Install Dependencies	SonarQube Analysis	Build React Application	Build Docker Image	DockerHub Login	Push Docker Image	Deploy To Kubernetes	Verify Kubernetes Deployment	Declarative: Post Actions
1s	14s	26s	13s	1min 3s	400ms	3s	580ms	303ms	355ms
785ms	6s	25s	8s	2min 6s	673ms	6s	1s	551ms	354ms
1s	22s	28s	18s	888ms	127ms	57ms	56ms	56ms	357ms

The last build attempt (#2) failed at 1:34 PM. The build log shows the following error:

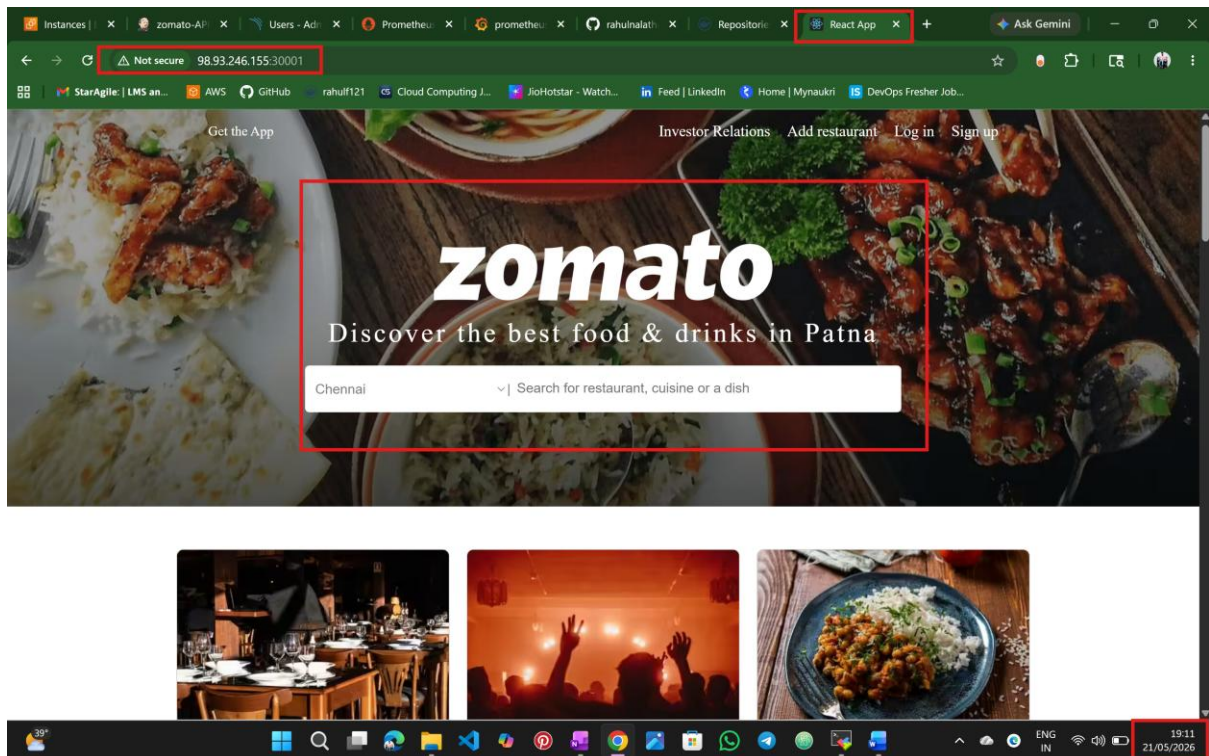
```
server-side processing: Success
```

The build status is 'Failed'.

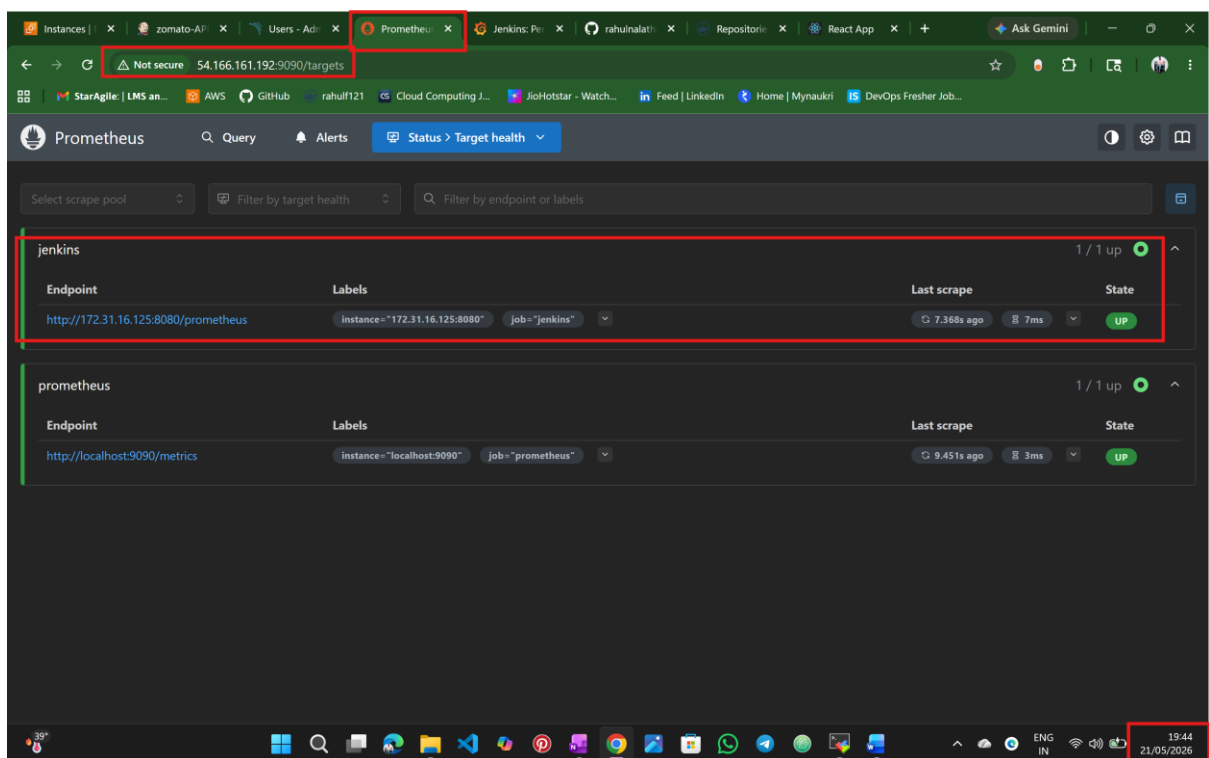
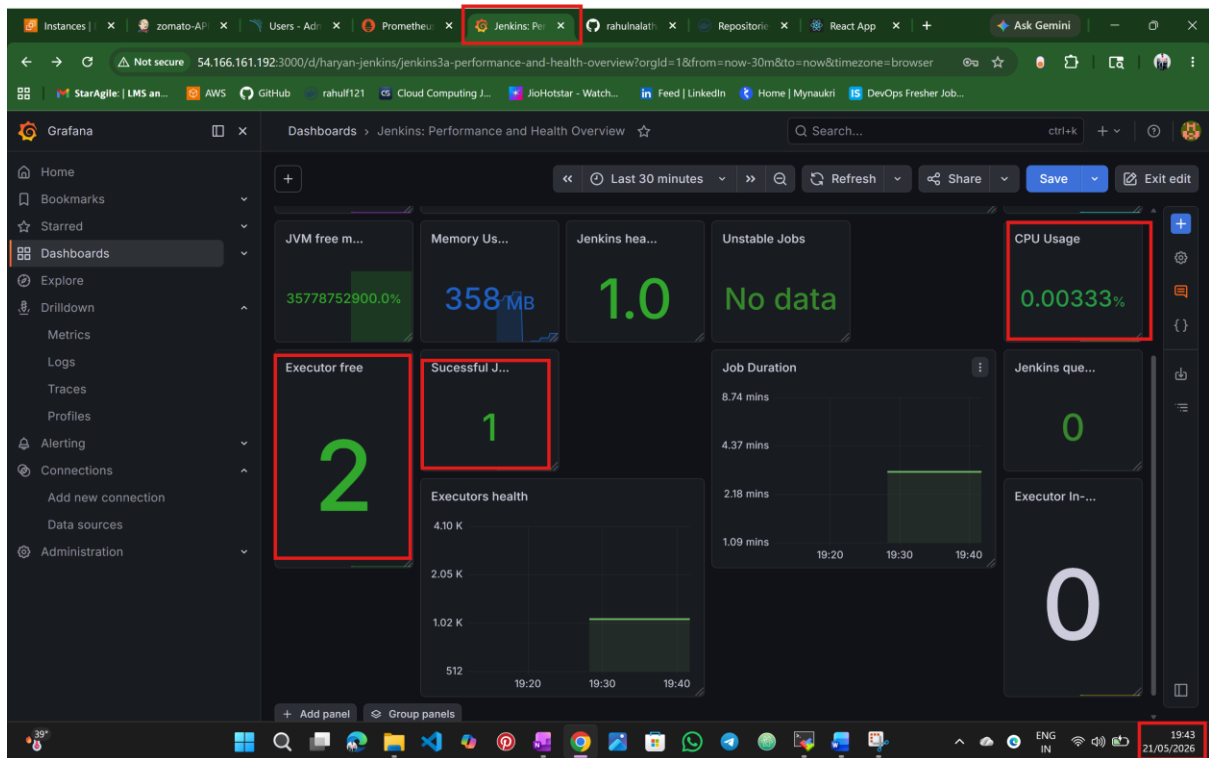
Step:12 we can see the through automation that docker image is pushed into docker hub



Step:13 finally we can access the application via k8s worker node ip address



Step:14 finally this is the visualization of Grafana.



Step:15 sonarQube code analysis

The screenshot shows the SonarQube web interface. The browser's address bar shows the URL `54.166.161.192:9000/projects`. The SonarQube navigation bar includes links for **Projects**, Issues, Rules, Quality Profiles, Quality Gates, and Administration. The **Projects** tab is active, displaying a list of projects. The project **zomato** is highlighted with a red box, showing a **Passed** status. The project details for **zomato** are shown below, including a table of metrics:

Metric	Value	Rating
Bugs	1	C
Vulnerabilities	0	A
Hotspots Reviewed	0.0%	E
Code Smells	0	A
Coverage	0.0%	F
Duplications	0.0%	F
Lines	1.3k	B

A warning message at the bottom of the interface states: "Embedded database should be used for evaluation purposes only. The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine."

Nalathathi Rahul